

BAB XII

TRANSACTION CONTROL LANGUAGE

12.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa memahami:

1. Mahasiswa mampu mendefinisikan dan memahami konsep dasar transaksi basis data sebagai satu kesatuan unit logis kerja (*logical unit of work*).
2. Mahasiswa mampu menjelaskan fungsi dan kegunaan dari perintah TCL: `COMMIT`, `ROLLBACK`, dan `SAVEPOINT`.
3. Mahasiswa mampu mengontrol aliran pemrosesan transaksi basis data untuk menjaga konsistensi data (*data consistency*) dan *read consistency*.
4. Mahasiswa mampu memahami konsep penguncian data (*locking*) baik secara implisit maupun eksplisit untuk mencegah konflik multi-user.

12.2 Dasar Teori

12.2.1 Konsep Transaksi Basis Data

Transaksi basis data merupakan konsep fundamental dalam setiap sistem manajemen basis data modern. Menurut Silberschatz et al. (2020), sebuah transaksi didefinisikan sebagai *logical unit of work* yang membundel satu atau lebih perintah *Data Manipulation Language* (DML) — yakni `INSERT`, `UPDATE`, `DELETE`, dan `MERGE` menjadi satu kesatuan yang tidak dapat dipisahkan. Prinsip ini sering dikenal dengan filosofi *all-or-nothing*: seluruh operasi di dalam sebuah transaksi harus berhasil dieksekusi secara penuh, atau tidak sama sekali.

Sebuah transaksi dimulai secara otomatis pada saat perintah DML pertama kali dieksekusi dalam sesi aktif. Transaksi tersebut berakhir apabila salah satu kondisi berikut terpenuhi: pengguna secara eksplisit mengeluarkan perintah `COMMIT` atau `ROLLBACK`; sebuah pernyataan DDL dieksekusi yang memicu *implicit commit*; sebuah pernyataan DCL dikeluarkan; atau pengguna keluar dari utilitas Oracle secara normal sehingga transaksi yang tertunda di-*commit* secara implisit. Setiap perubahan data yang dibuat selama transaksi berlangsung bersifat sementara hingga transaksi tersebut di-*commit* (Ramakrishnan & Gehrke, 2019).

12.2.2 Transaction Control Language (TCL)

Di dalam pengelolaan basis data (seperti Oracle), kontrol transaksi dilakukan melalui serangkaian perintah terintegrasi untuk menjaga konsistensi data. Proses ini dimulai ketika pengguna melakukan perubahan data dan memutuskan untuk menyimpannya secara permanen menggunakan perintah `COMMIT`. Ketika perintah ini dieksekusi, seluruh perubahan tertunda dalam transaksi yang berjalan akan disahkan dan ditulis secara permanen ke dalam basis data, bersamaan dengan dilepaskannya kunci (lock) pada baris yang terpengaruh. Namun, jika terjadi kesalahan atau pengguna berubah pikiran sebelum data tersebut di-*commit*, perintah `ROLLBACK` dapat digunakan untuk membatalkan seluruh perubahan yang belum tersimpan tersebut. Mekanisme `ROLLBACK` ini bekerja dengan cara menuliskan kembali versi lama data dari *undo segment* ke dalam tabel, lalu melepaskan semua kunci yang ada sehingga data kembali ke kondisi semula.

Untuk transaksi yang kompleks dan panjang, pengguna dapat memanfaatkan perintah `SAVEPOINT` guna menciptakan titik penanda (*marker*) di dalam aliran transaksi yang sedang berjalan. Kehadiran *savepoint* ini efektif untuk membagi satu transaksi besar menjadi segmen-segmen yang lebih kecil. Fleksibilitas ini kemudian didukung oleh perintah `ROLLBACK TO SAVEPOINT`, yang memungkinkan pengguna untuk membatalkan seluruh perubahan data yang terjadi *setelah* titik penanda tertentu dibuat, tanpa harus menggugurkan keseluruhan transaksi. Dengan kombinasi ini, transaksi tidak langsung berakhir, melainkan hanya mundur secara selektif ke kondisi tepat saat *savepoint* tersebut diciptakan, sementara perubahan yang dilakukan sebelum *savepoint* tetap dipertahankan.

12.3 Hasil dan Pembahasan

Pada bagian ini akan ditampilkan hasil dari seluruh percobaan *query* yang telah dipelajari pada modul *section* 18, beserta pembahasan teknis mendalam mengenai *output* yang dihasilkan dari setiap perintah TCL yang dijalankan.

1. *Query*:

```
UPDATE copy_departments
SET manager_id = 101
WHERE department_id = 60;
```

Hasil:

```
SQL> UPDATE copy_departments
2 SET manager_id = 101
3 WHERE department_id = 60;

1 row updated.
```

Gambar 12.1 *Update manager id departemen 60*

Penjelasan:

Query ini digunakan untuk mengubah data pada kolom `manager_id` menjadi 101 khusus untuk baris yang memiliki `department_id` bernilai 60 di dalam tabel `copy_departments`. Perlu dipahami bahwa eksekusi perintah `UPDATE` ini secara otomatis memulai sebuah transaksi baru dalam sesi aktif jika belum ada transaksi yang berjalan. Perubahan yang dihasilkan bersifat sementara dan hanya terlihat oleh sesi pengguna yang menjalankannya; pengguna lain masih akan melihat data lama dari *undo segment* hingga transaksi ini di-*commit* secara resmi. Penggunaan klausa `WHERE` yang tepat pada *query* ini menjadi sangat kritis untuk mencegah modifikasi massal yang tidak disengaja.

2. *Query*:

```
SAVEPOINT one;
```

Hasil:

```
SQL> SAVEPOINT one;  
  
Savepoint created.
```

Gambar 12.2 Pembuatan *savepoint one*

Penjelasan:

Query ini berfungsi untuk menciptakan titik penanda atau titik simpan bernama *one* dalam sesi transaksi aktif. Layaknya sebuah pos pemeriksaan dalam sebuah perjalanan, *SAVEPOINT* merekam status transaksi saat ini sebagai batas referensi. Titik penanda ini nantinya dapat digunakan sebagai batas tujuan jika ingin membatalkan perintah SQL yang dieksekusi setelah titik ini, tanpa menghapus atau membatalkan perubahan yang telah terjadi sebelum titik penanda ini dibuat. Dalam konteks praktikum ini, *SAVEPOINT one* ditetapkan tepat setelah operasi *UPDATE* pada langkah pertama, sehingga perubahan *manager_id* terlindungi dari kemungkinan *rollback* parsial di masa mendatang.

3. *Query*:

```
INSERT INTO copy_departments (department_id,  
department_name, manager_id, location_id)  
VALUES (130, 'Estate Management', 102, 1500);
```

Hasil:

```
SQL> INSERT INTO copy_departments (department_id, department_name, manager_id, location_id)  
2  VALUES (130, 'Estate Management', 102, 1500);  
  
1 row created.
```

Gambar 12.3 *Insert* data departemen baru

Penjelasan:

Query ini digunakan untuk menyisipkan atau menambah satu baris data baru ke dalam tabel `copy_departments` dengan rincian data: `department_id` bernilai 130, `department_name` berisi 'Estate Management', `manager_id` bernilai 102, dan `location_id` bernilai 1500. Operasi `INSERT` ini termasuk dalam transaksi yang sama dengan `UPDATE` sebelumnya. Data yang disisipkan ini juga bersifat sementara dan belum tersimpan secara permanen karena belum ada perintah `COMMIT` yang dikeluarkan. Penting untuk dicatat bahwa perubahan ini terjadi setelah `SAVEPOINT one` dibuat, sehingga perubahan ini rentan terhadap pembatalan parsial melalui `ROLLBACK TO SAVEPOINT`.

4. *Query*:

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:

```
SQL> ROLLBACK TO SAVEPOINT one;  
Rollback complete.
```

Gambar 12.4 *Rollback ke savepoint one*

Penjelasan:

Query ini digunakan untuk membatalkan semua perubahan data yang terjadi setelah penanda `one` dibuat. Eksekusi perintah ini menyebabkan Oracle *Database* mengembalikan kondisi data ke keadaan tepat saat `SAVEPOINT one` ditetapkan. Akibatnya, proses pengisian data departemen 'Estate Management' pada langkah ke-3 otomatis dibatalkan dan baris tersebut dihapus dari tampilan sesi aktif, sedangkan perubahan `manager_id` pada langkah ke-1 tetap aman dan tidak terpengaruh karena terjadi sebelum titik penanda. Mekanisme ini sangat berguna dalam skenario di mana seorang DBA menyadari adanya kesalahan input setelah beberapa operasi, namun tidak ingin mengulang seluruh transaksi dari awal.

5. *Query*:

```
COMMIT;
```

Hasil:

```
SQL> COMMIT;  
  
Commit complete.
```

Gambar 12.5 *Commit* transaksi pertama

Penjelasan:

Query ini berfungsi untuk menyimpan seluruh rangkaian perubahan data yang sah secara permanen ke dalam basis data. Ketika `COMMIT` dieksekusi, Oracle *Database* melakukan beberapa hal sekaligus: perubahan `manager_id` pada langkah ke-1 resmi ditulis secara permanen dan tidak dapat dibatalkan lagi; transaksi yang sedang berjalan dinyatakan berakhir; seluruh kunci (locks) yang diterapkan pada baris-baris yang termodifikasi dilepaskan sehingga pengguna lain dapat mengaksesnya; serta data yang telah di-*commit* kini menjadi tampak bagi seluruh sesi pengguna lain yang melakukan *query* `SELECT` pada tabel tersebut. Data 'Estate Management' yang sebelumnya telah di-*rollback* tidak ikut tersimpan karena memang telah dibatalkan sebelum `COMMIT` ini dijalankan.

6. *Query*:

```
UPDATE copy_departments SET department_id = 140;
```

Hasil:

```
SQL> UPDATE copy_departments SET department_id = 140;  
10 rows updated.
```

Gambar 12.6 *Update massal department id*

Penjelasan:

Query ini digunakan untuk mengubah nilai kolom `department_id` menjadi 140 untuk seluruh baris yang ada di dalam tabel `copy_departments` tanpa terkecuali. Ketidakhadiran klausa `WHERE` pada *query* ini merupakan sebuah kesalahan kritis yang menyebabkan seluruh baris dalam tabel termodifikasi secara massal sebuah kondisi yang berpotensi merusak integritas data secara fatal. Eksekusi *query* ini sekaligus memulai transaksi baru setelah `COMMIT` di langkah ke-5. Perubahan massal ini masih bersifat sementara (pending) dalam sesi aktif dan belum tersimpan permanen, sehingga masih dapat dibatalkan seluruhnya dengan perintah `ROLLBACK`.

12.4 Latihan

Latihan yang telah dilakukan pada praktikum ini adalah sebagai berikut:

1. Menggunakan kombinasi perintah `INSERT` dan `ROLLBACK` untuk memahami mekanisme pembatalan total transaksi.

Query:

```
INSERT INTO mahasiswa (nim, nama, prodi)
VALUES (9067893822, 'Wan Eva Yuliani',
'Akuntansi');
ROLLBACK;
```

Hasil:

```
SQL> INSERT INTO mahasiswa (nim, nama, prodi)
2  VALUES (9067893822, 'Wan Eva Yuliani', 'Akuntansi');

1 row created.

SQL> ROLLBACK;

Rollback complete.
```

Gambar 12.7 *Insert data mahasiswa dan rollback transaksi*

Penjelasan:

Query ini terdiri dari dua perintah yang dieksekusi secara berurutan dalam satu blok transaksi. Pertama, perintah `INSERT` digunakan untuk menyisipkan data mahasiswa baru bernama 'Wan Eva Yuliani' dengan NIM 9067893822 dan program studi 'Akuntansi' ke dalam tabel `mahasiswa`. Namun, segera setelah itu, perintah `ROLLBACK` dieksekusi yang membatalkan secara total seluruh perubahan yang ada dalam transaksi berjalan. Akibatnya, data mahasiswa tersebut tidak jadi disimpan ke dalam basis data. Yang lebih signifikan, perintah `ROLLBACK` ini juga sekaligus membatalkan perubahan massal `department_id` pada langkah ke-6 yang belum sempat di-*commit*, sehingga kondisi basis data dikembalikan seperti semula setelah perintah `COMMIT` terakhir pada langkah ke-5. Skenario latihan ini menegaskan betapa krusialnya manajemen transaksi: sebuah `ROLLBACK` tunggal mampu membatalkan sekaligus seluruh rangkaian perubahan yang tertunda dalam satu sesi.

12.5 Kesimpulan

Berdasarkan hasil praktikum yang telah dilaksanakan pada modul ini, dapat disimpulkan bahwa pengendalian aliran transaksi menggunakan TCL (Transaction Control Language) seperti `COMMIT`, `ROLLBACK`, dan `SAVEPOINT` sangat krusial bagi integritas data bisnis. Melalui prinsip *all-or-nothing*, sistem memastikan bahwa sekelompok operasi DML yang saling berkaitan akan berjalan secara utuh. Hal ini mencegah terjadinya kondisi data yang setengah jadi (*inconsistent state*) yang dapat merusak keandalan sistem informasi secara keseluruhan.

Di samping itu, penggunaan `SAVEPOINT` memberikan fleksibilitas tinggi dalam proses pengelolaan transaksi yang kompleks. Kemampuan untuk membatalkan perubahan data secara parsial hingga titik penanda tertentu tanpa harus mengulang seluruh transaksi dari awal terbukti sangat efisien, terutama dalam menangani kesalahan input yang baru terdeteksi di tengah-tengah proses—seperti yang didemonstrasikan melalui pembatalan `INSERT` pada departemen 'Estate Management'.

Terakhir, sistem Oracle secara otomatis menerapkan *read consistency* dan mekanisme *locking* guna menjamin keamanan data dalam lingkungan *multi-user*. Melalui *undo segment*, *read consistency* memastikan para pembaca (readers) selalu mendapatkan visualisasi data yang konsisten sebelum data tersebut resmi di-commit. Sementara itu, mekanisme *implicit locking* bertugas mencegah konflik destruktif antar-pengubah (writers) yang mengakses baris yang sama secara bersamaan, sehingga integritas data tetap terjaga dalam setiap kondisi operasional.